Sparse Jacobian Computation in Futhark

Exploiting Known Sparsity Patterns Using Graph Coloring

Elias Fontain Smedegaard

Department of Computer Science
University of Copenhagen

BSc Thesis Defense
June 17, 2026

Outline

Problem and goal

Central problem

Given a differentiable Futhark function and a user-provided Jacobian sparsity pattern, can we compute the structurally nonzero Jacobian entries more efficiently than by computing the full dense Jacobian?

$$\text{known sparsity pattern} \implies \text{avoid computing known zeros}$$

- Design and implement a sparse Jacobian library in Futhark
- Use graph coloring to compress forward- and reverse-mode automatic differentiation (AD)
- Validate correctness and benchmark against dense baselines

Known sparsity in the running example

Consider the function

$$f(x_0, x_1, x_2, x_3) = \begin{bmatrix} \sin(x_0) + x_2^3 \\ x_1 x_3 \\ x_2^2 + 3x_3 \\ x_0 x_1 \end{bmatrix}.$$

Its Jacobian is

$$J_f(x) = \begin{bmatrix} \cos(x_0) & 0 & 3x_2^2 & 0 \\ 0 & x_3 & 0 & x_1 \\ 0 & 0 & 2x_2 & 3 \\ x_1 & x_0 & 0 & 0 \end{bmatrix}.$$

This example will be used throughout the first part of the presentation.

Motivation: Dense Jacobians can be wasteful

Dense Jacobian

$$J_f(x) = \begin{bmatrix} \cos(x_0) & 0 & 3x_2^2 & 0 \\ 0 & x_3 & 0 & x_1 \\ 0 & 0 & 2x_2 & 3 \\ x_1 & x_0 & 0 & 0 \end{bmatrix}$$

Known sparsity pattern

$$S = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

Gray entries are **structurally zero**: they are known to be zero from the dependency structure.

16 entries $\Rightarrow$ 8 may be nonzero

Goal: compute only the structurally nonzero entries

Dense baseline

A full Jacobian can be recovered one direction at a time.

Forward mode

$$\text{JVP} = J_f(x)v$$

one Jacobian-vector product per column

n columns $\Rightarrow$ n calls

Reverse mode

$$\text{VJP} = u^T J_f(x)$$

one vector-Jacobian product per row

m rows $\Rightarrow$ m calls

For the running example, $m = n = 4$, so both dense baselines use 4 AD calls.

JVP/VJP interface follows the Futhark AD interface, see Schenck et al. (2022).

Sparse compression: the main idea for JVP

$$J_f(x) = \begin{bmatrix} \cos(x_0) & 0 & 3x_2^2 & 0 \\ 0 & x_3 & 0 & x_1 \\ 0 & 0 & 2x_2 & 3 \\ x_1 & x_0 & 0 & 0 \end{bmatrix}$$

- Columns in the same group must not have nonzeros in the same row
- Here we can group the columns as

$$C_0 = \{0, 3\}, \qquad C_1 = \{1, 2\}$$

Key observation

Same group (color)

$\Downarrow$

no row conflicts

Benefit

4 columns

$\Downarrow$

2 color groups

Compressed JVPs

Instead of one seed vector per column, we use one seed vector per color group.

Color groups

$$C_0 = \{0, 3\}, \qquad C_1 = \{1, 2\}$$

Compressed seed vectors

$$s_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \qquad s_1 = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Using these seeds, we perform one JVP per color group and obtain two compressed seed vectors:

$$z_0 = J_f(x)s_0, \qquad z_1 = J_f(x)s_1$$

$$Z = \begin{bmatrix} z_0 & z_1 \end{bmatrix} = \begin{bmatrix} \cos(x_0) & 3x_2^2 \\ x_1 & x_3 \\ 3 & 2x_2 \\ x_1 & x_0 \end{bmatrix}$$

The compressed Jacobian has one column per color group.

Graph coloring view of the running example (column coloring)

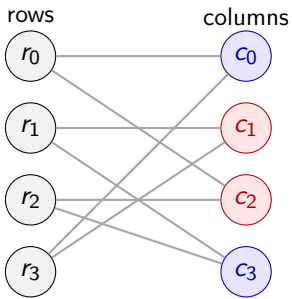
- Build a bipartite graph from the sparsity pattern
- Color the column vertices
- Shared row neighbor means different colors

Notation

$$r_i = \text{row } i$$

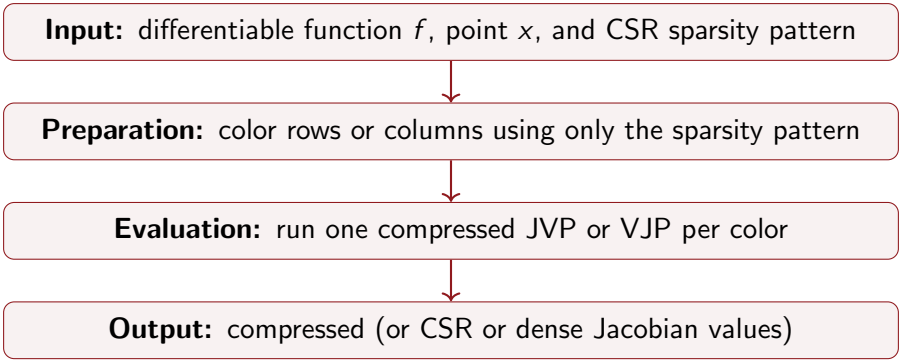
$$c_j = \text{column } j$$

$$(r_i, c_j) \in E \quad \text{if } S_{ij} = 1$$



Only the column side is colored, so this is a partial distance-2 coloring.

Sparse Jacobian pipeline



The preparation phase depends only on the sparsity pattern, so it can be reused for several input points if the sparsity pattern stays the same.

CSR sparsity representation

The library stores the sparsity pattern using row-wise CSR arrays.

$$S = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

$$\texttt{row\_offs} = [0, 2, 4, 6, 8]$$

$$\texttt{row\_idx} = [0, 2,\ \ 1, 3,\ \ 2, 3,\ \ 0, 1]$$

Meaning

$$\texttt{row\_idx}[\texttt{row\_offs}[i] : \texttt{row\_offs}[i+1]]$$

stores the nonzero column indices in row i.

Example:

$$\text{row } 0: \quad \texttt{row\_idx}[0:2] = [0, 2]$$
$$\text{row } 1: \quad \texttt{row\_idx}[2:4] = [1, 3]$$

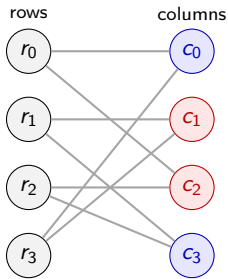
This stores only the structurally nonzero positions instead of the full $m \times n$ matrix.

Two sparse views of the same graph

The implementation stores both directions of the bipartite graph.

Row-wise view

row	columns
r_0	c_0, c_2
r_1	c_1, c_3
r_2	c_2, c_3
r_3	c_0, c_1



Column-wise view

column	rows
c_0	r_0, r_3
c_1	r_1, r_3
c_2	r_0, r_2
c_3	r_1, r_2

Together, the two views store the bipartite graph needed for coloring and CSR reconstruction.

Coloring algorithms in the implementation

Coloring is a preprocessing step. I implemented two approaches.

D2

Greedy Partial Distance-2 coloring

Sequential greedy loop

A greedy coloring method that assigns colors so distance-2 neighbors get different colors.

Used as the robust default in the library.

Based on Gebremedhin et al. (2005).

VS.

BGPC

Optimistic vertex-based Bipartite Graph Partial Coloring

Optimistic work queue

Conflict removal between iterations

More parallel alternative

BGPC inspired by Taş et al. (2017).

The evaluation compares both coloring quality and coloring runtime.

JVP and VJP pipelines

After coloring, the implementation uses one of two sparse automatic differentiation pipelines.

Column colors → JVP

column colors $\Longrightarrow$ $s_0, \ldots, s_{p-1}$

$s_k \xrightarrow{\texttt{jvp } f\ x} z_k = J_f(x)s_k$

p colors $\Longrightarrow$ p JVP calls

Row colors → VJP

row colors $\Longrightarrow$ $\bar{s}_0, \ldots, \bar{s}_{q-1}$

$\bar{s}_k \xrightarrow{\texttt{vjp } f\ x} \bar{z}_k = \bar{s}_k^T J_f(x)$

q colors $\Longrightarrow$ q VJP calls

The compressed outputs are unpacked using the sparsity pattern and the color arrays.

Automatic mode and reuse

The automatic pipeline chooses between the JVP and VJP pipelines during preparation.

$$\texttt{use\_jvp} = \texttt{num\_col\_colors} \leq \texttt{num\_row\_colors}$$

- If column coloring gives fewer colors, use the JVP pipeline
- If row coloring gives fewer colors, use the VJP pipeline
- Ties are handled deterministically by choosing JVP
- Automatic mode returns CSR output, because compressed JVP and VJP outputs have different shapes
- The prepared colorings can be reused for several input points with the same sparsity pattern

Library API

The library provides three main functions.

Sparse Jacobian calls

`jac_jvp_compressed_from_csr`

`jac_vjp_compressed_from_csr`

`jac_auto_csr_from_csr`

`CSR pattern` → `coloring` → `compressed AD` → `output`

Optional: the coloring functions for the D2 and BGPC algorithms are also given directly: `partial_d2_color_*` and `vv_color_*`.

Complexity intuition (CSR input $\rightarrow$ compressed output)

Let p be the number of column colors and q the number of row colors.

Sparse JVP pipeline

Work: $O(W(\mathrm{color}) + pn + p \cdot W(f))$

Span: $O(S(\mathrm{color}) + \log n + S(f))$

Sparse VJP pipeline

Work: $O(W(\mathrm{color}) + qm + q \cdot W(f))$

Span: $O(S(\mathrm{color}) + \log m + S(f))$

Sparse compression helps when $p \ll n$ or $q \ll m$ and the saved AD work outweighs coloring overhead.

Evaluation questions

The evaluation asks four main questions:

1. Are the sparse Jacobian results correct?
2. When is the sparse pipeline faster than dense baselines?
3. How much does coloring affect total runtime?
4. How does my general sparse pipeline compare to benchmark-specific Jacobian code?

Evaluation setup

Goals

- Validate sparse results against dense Jacobians
- Measure speedups over dense CPU baselines
- Separate coloring cost from sparse evaluation cost

Benchmarks

- `Banded5`
- `Stencil`
- `BA` from ADBench/GradBench
- `HT` from ADBench/GradBench

$$\text{speedup} = \frac{T_{\text{Dense CPU}}}{T_{\text{method}}}$$

Benchmarks were run with `futhark bench --runs=10` through Slurm on Hendrix, using the multicore CPU backend and the CUDA backend on an NVIDIA A100 GPU.

Correctness testing

Sparse outputs are compared against dense Jacobians restricted to the sparsity pattern.

$$|J_{\text{sparse}}(i,j) - J_{\text{dense}}(i,j)| \leq 10^{-9} \quad \text{for all } S_{ij} = 1$$

- CSR construction tested by exact pattern reconstruction, and coloring by exact integer and validity checks
- Sparse JVP, VJP, and automatic pipelines tested against dense baselines
- BA and HT wrappers tested on small benchmark instances

All correctness tests pass

Benchmark overview

The evaluation uses four benchmark problems.

Synthetic benchmarks

- `Banded5`
  banded pattern with five inputs per output
- `Stencil`
  2D five-point stencil

ADBench/GradBench benchmarks

- `BA`
  Bundle Adjustment
- `HT`
  Hand Tracking

Synthetic cases have known sparsity patterns with five nonzeros per row.
ADBench/GradBench benchmarks test more realistic automatic differentiation workloads.

BA and HT benchmark structures are based on ADBench/GradBench: https://github.com/gradbench/gradbench

Synthetic benchmarks

Both synthetic benchmarks have controlled sparsity with five nonzeros per row.

`Banded5`

One-dimensional band structure with wrap-around boundaries.

Each output depends on a center input and two neighbors on each side.

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

`Stencil`

Two-dimensional five-point stencil with wrap-around boundaries.

Each output depends on the center grid point and its north, south, east, and west neighbors.

$$\begin{matrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{matrix}$$

D2 JVP/VJP: `Banded5` [5,5,5]/[5,4,5], `Stencil` [9,7,9]/[9,7,9].
BGPC JVP/VJP: `Banded5` [5,5,5]/[5,4,5], `Stencil` [9,9,9]/[9,9,9].

Results: structured benchmarks, JVP

End-to-end JVP speedups relative to Dense CPU.

Case	Size	Dense GPU	D2 CPU	BGPC CPU	BGPC GPU
Banded5	512×16384	4.5×	**30.6**×	4.4×	25.1×
Banded5	1024×32768	5.1×	**120.0**×	16.3×	114.3×
Banded5	2048×65536	5.2×	217.9×	47.9×	**438.4**×
Stencil	4096×4096	17.2×	**33.3**×	0.7×	0.9×
Stencil	9216×9216	12.7×	**102.1**×	0.3×	0.9×
Stencil	16384×16384	11.1×	**208.5**×	0.4×	1.4×

D2 CPU is consistently strong. BGPC GPU is fastest on large `Banded5`, but not on `Stencil`.

Results: structured benchmarks, VJP

End-to-end VJP speedups relative to Dense CPU.

Case	Size	Dense GPU	D2 CPU	BGPC CPU	BGPC GPU
Banded5	2048×4096	**2.7**×	2.5×	0.04×	0.02×
Banded5	4096×8192	2.5×	**7.0**×	0.03×	0.03×
Banded5	8192×16384	2.7×	**26.3**×	0.03×	0.06×
Stencil	4096×4096	1.8×	**1.9**×	0.09×	0.15×
Stencil	9216×9216	1.6×	**5.6**×	0.07×	0.18×
Stencil	16384×16384	1.4×	**45.5**×	0.09×	0.3×

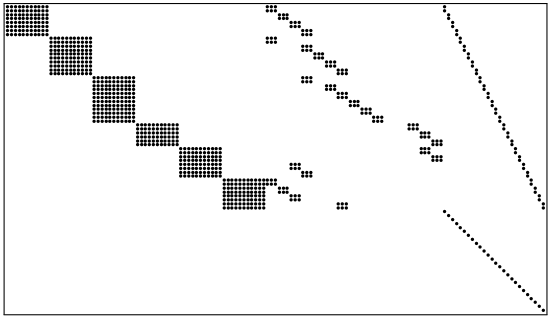
VJP shows the same robust D2 pattern, while BGPC performs very poorly across these cases.

ADBench/GradBench benchmarks (Figures below reproduced from Šrajer et al. (2018))

BA: Bundle Adjustment

Each observation depends on one camera, one 3D point, and one weight.

$$3N \times (11C + 3M + N)$$

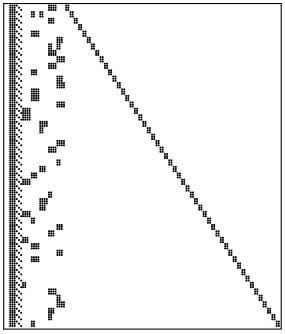


15 column colors in tested cases.

HT: Hand Tracking

Motion parameters affect all residuals. Local U variables affect only their own residual.

$$3N \times (26 + 2N)$$



28 column colors in tested cases

Results: BA and HT

End-to-end JVP speedups relative to Dense CPU.

Case	Size	Dense GPU	D2 CPU	BGPC CPU	BGPC GPU
BA	6144×2784	**22.9**×	15.2×	2.5×	1.4×
BA	12288×5200	17.6×	**22.3**×	4.6×	2.4×
BA	24576×9664	10.9×	**21.8**×	5.4×	3.2×
BA	36864×14128	8.4×	**29.4**×	7.5×	4.1×
HT	1536×1050	0.7×	**1.4**×	0.7×	0.04×
HT	3072×2074	0.7×	**3.0**×	1.2×	0.07×
HT	6144×4122	0.7×	**6.8**×	2.5×	0.14×
HT	12288×8218	0.6×	**15.2**×	4.7×	0.3×

D2 CPU gives consistent sparse speedups. Dense GPU wins only on the smallest BA case, while GPU sparse variants are more mixed.

Why? Pipeline breakdown

CPU timings for the JVP pipeline show where the sparse runtime is spent.

Case	Size	D2 full	D2 color	D2 precolored	BGPC color
Banded5	2048×65536	1085	405	467	4301
Stencil	16384×16384	972	505	444	654910
BA	36864×14128	6831	6261	1562	25563
HT	12288×8218	53996	8525	40801	135152

Times are in microseconds. Precolored means that the color assignment is supplied as input.

- D2 usually wins because its coloring is much cheaper
- BGPC often finds similar color counts, but coloring can dominate runtime
- For HT, the compressed evaluation itself also becomes expensive

Comparison with benchmark-specific Jacobian code

Speedups relative to Dense CPU on the largest BA and HT cases.

Case	Dense CPU	My D2 CPU	ADBench CPU	ADBench GPU
BA	210175	19.1×	49.0×	**2040.5**×
HT	903892	16.7×	18.7×	**177.1**×

For BA, My D2 CPU uses CSR output. For HT, it uses compressed output.

- On HT, my D2 CPU pipeline is close to ADBench CPU
- On GPU, problem-specific code is much faster

Discussion: main takeaways

- My sparse pipelines are effective when the number of colors is small compared with the Jacobian dimension
- D2 CPU is the most robust sparse variant in the benchmarks
- BGPC GPU performs very well on `Banded5` JVP, but is much less consistent overall
- Coloring time matters most in my benchmarks, not the amount of colors found
- Problem-specific Jacobian code can still be much faster, especially on GPU

Limitations and future work

- Sparsity patterns must be provided by the user
- CSR reconstruction cost is not included in most of the main compressed benchmarks
- GPU performance is still inconsistent for the sparse pipelines
- Future work: automatic sparsity-pattern discovery
- Future work: more GPU-friendly coloring or hybrid CPU/GPU execution
- Future work: additional sparse Jacobian methods such as cross-country algorithms

Conclusion

- I implemented a Futhark library for sparse Jacobian computation from a known sparsity pattern
- The library uses CSR patterns, graph coloring, and compressed JVP/VJP evaluation
- Correctness tests validate the sparse pipelines against dense Jacobians
- Benchmarks show large speedups when the color count is small
- D2 CPU was the most robust sparse variant in the evaluation

When the sparsity pattern is known, sparse compression can make Jacobian computation much faster in Futhark.

Thank you for listening!

Sources:

- Gebremedhin, Manne, and Pothen. *What Color Is Your Jacobian?* SIAM Review, 2005.
- Taş, Kaya, and Saule. *Greed is Good: Optimistic Algorithms for Bipartite-Graph Partial Coloring on Multicore Architectures*, 2017.
- Schenck et al. *AD for an Array Language with Nested Parallelism*, 2022.
- Šrajer, Kukelova, and Fitzgibbon. *A Benchmark of Selected Algorithmic Differentiation Tools*, 2018.